Структуризація паралелізму операцій в комп’ютерних системах

Головні ідеї при розпаралелюванні операцій

- Головними ідеями при розпаралелюванні операцій в обчислювальних системах є ідея *конвейєризації* обчислень потоку даних та ідея *власне паралелізму* (або *справжнього паралелізму*) – тобто незалежного (і одночасного) виконання операцій на різних пристроях обчислювальної машини.
- Використання цих двох ідей в чистому вигляді або в їх поєднанні привели до таких *видів* паралелізму в архітектурі ЕОМ:
  - конвейєрна обробка;
  - функціональна обробка;
  - векторно-матрична обробка;
  - мультипроцесорна обробка.

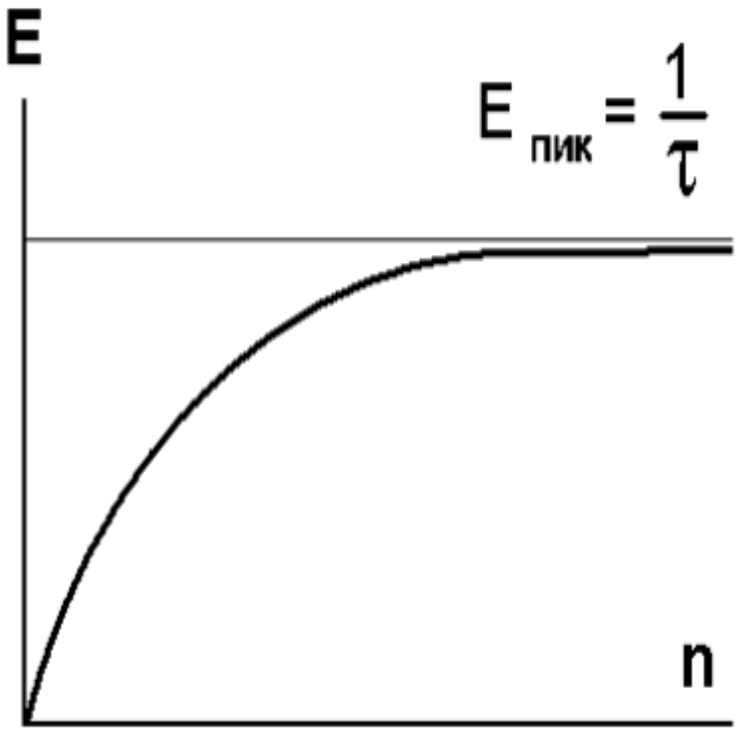
Власне паралелізм

- Власне паралелізм передбачає наявність *p* *однакових пристроїв* для обробки даних і алгоритм, що дозволяє виконувати на кожному з них *незалежну частину* обчислень.
- В кінці обробки часткові дані збираються разом для отримання остаточного результату. У цьому випадку (нехтуючи накладними витратами на отримання та збереження даних) отримаємо *прискорення процесу в* *p* *разів*.
- Проте далеко не кожен алгоритм може бути успішно розпаралелений таким способом (природною умовою розпаралелювання є обчислення *незалежних* частин вихідних даних за однаковими - або подібним – процедурам).
- Ітераційний або рекурсивний характер обчислень викликають найбільші проблеми при розпаралелюванні)

Конвеєрний спосіб

- Конвеєрний спосіб (або, 'принцип водогону') при виконанні команд імітує роботу виробничого конвеєра, на якому виріб послідовно проходить ряд робочих місць, де на кожному з них над виробом проводиться нова операція.
- Ефект прискорення досягається за рахунок *одночасної* обробки ряду виробів на *всіх* робочих місцях.
- Конвеєри можуть бути *скалярними* або *векторними* пристроями (в останньому випадку можуть бути використані обробні векторні команди).
- У разі довжини конвеєра l час обробки n незалежних операцій складе $l + n - 1$.
- При використанні такого пристрою для обробки одної порції вхідних даних буде потрібно час $l \times n$, і тільки для множини порцій отримаємо прискорення обчислень, близьке до l.

Ефект конвеєризації



- **Швидкодія *E* конвеєрного пристрою визначається як:**

$$E = \frac{n}{t} = \frac{1}{[\tau + (\sigma + l - 1)\frac{\tau}{n}]}$$

- **де *n* – кількість виконаних операцій,**
- ***t* – час їх виконання,**
- **τ – час такту роботи комп'ютера,**
- **σ – час ініціалізації команди.**

Умови успішної конвейєризації розпаралелювання операцій

1. Передбачається наявність декількох місць (стадій) обробки, через які *послідовно проходить кожний елемент* оброблюваного потоку даних. *Чим довший конвейєр*, тобто більша кількість його стадій – *тим вища ступінь розпаралелювання* операцій.
2. Час обробки елемента даних на кожному місці обробки – приблизно однаковий, тоді втрати від розпаралелювання обчислень – мінімальні. Чим довший вхідний потік даних – *тим вища ступінь розпаралелювання* операцій.

- Використання цього вид паралелізму в арифметичному і керуючому пристроях ЕОМ *стало хронологічно першим і найпоширенішим*, особливо в науково-технічних застосуваннях, що характеризуються високою регулярністю обчислень.

Види: функціональний паралелізм

- Передбачає надання *можливості одночасного виконання різних функцій обробки* (наприклад, логічних та арифметичних) декільком незалежним функціональним пристроям процесора.
- Традиційним прикладом цього виду паралелізму є множинне застосування *кількох арифметико-логічних пристроїв*.
- Іншим класичним прикладом використання функціонального паралелізму є наявність *співпроцесора* в архітектурі персонального комп'ютера для виконання операцій з плаваючою комою.

Види: векторно-матрична обробка

- Це *поєднання* конвейєрного та функціонального паралелізму.
- Важливою ознакою при цьому є наявність множини ідентичних процесорних елементів з єдиною системою керування, що взаємодіють між собою через спільне поле пам'яті.
- Важливою віхою в реалізації цього виду паралелізму стала перша в світі модель *суперкомп'ютерної системи Cray-1* (1976 р.), процесор якої містив 12 функціональних пристроїв, і всі конвейєрні, а також 8 векторних регістрів, кожний з яких міг містити 64 64-розрядних чисел у форматі з плаваючою комою, і показував рекордну на свій час продуктивність 130 Мфлопс.

Види: мультипроцесорна обробка (або, справжній паралелізм)

- На відміну від зазначених вище видів паралелізму, реалізується *множиною повнофункціональних процесорів*, кожний з яких є повноцінним комп’ютером хіба що зі спеціалізованими засобами зв’язку між процесорами та засобами вводу/виводу.
- Справжній паралелізм, на відміну від конвейєрного, не вимагає організації вхідних даних у потік і характеризується *незалежним і одночасним виконанням* операцій над різними даними.

Рівні паралелізму в архітектурі комп’ютерів

- Рівень структуризації одиниць даних та обчислень, на якому відбувається розпаралелювання обробки в паралельних обчислювальних системах, визначає *рівень паралелізму* обчислень у цих системах. До відомих рівнів паралелізму належать:
  - рівень завдань;
  - рівень програм;
  - рівень команд;
  - рівень розрядів.
- Певною мірою рівень паралелізму обчислень, що реалізується в паралельній обчислювальній системі, залежить від виду паралелізму, що закладений в архітектурі системи.

Рівень команд: суперскалярні процесори

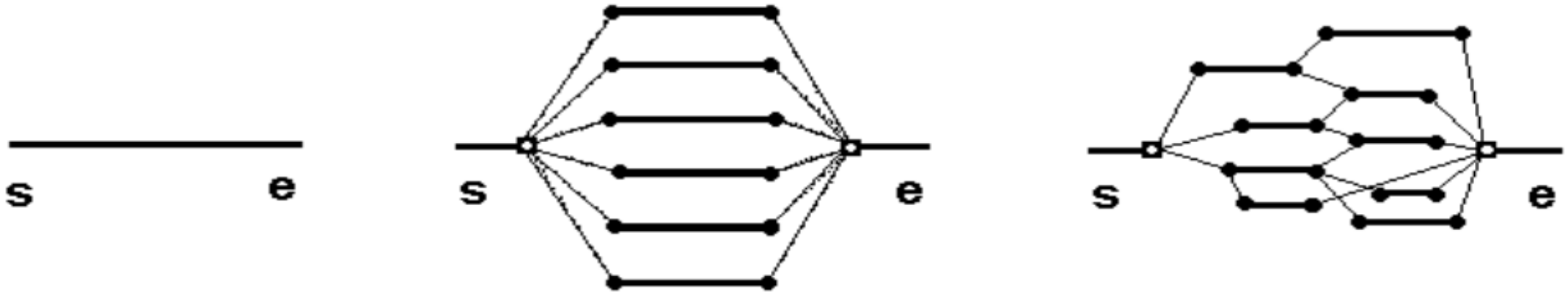
- *Суперскалярні* IPL-системи (Instruction Level Parallelism), реалізують прихований від користувача паралелізм. Програми для таких процесорів не містить вказівок на можливу паралельність в обробці, і забезпечують динамічне завантаження паралельних ділянок програм і їх виконання апаратурою мікропроцесора *без втручання програміста.*
- Архітектура таких мікропроцесорів (типовий приклад – процесори серії DEC Alpha) розрахована на виконання якомога більшої кількості інструкцій *одночасно* (часто у порядку, відмінному від вихідної послідовності в програмі).
- Подальшим розвитком суперскалярної архітектури є мультипотокова (мультитредова), де програма розбивається (апаратними та програмними засобами) на сукупність тредів, за виконання яких відповідає *неперервна область* динамічної послідовності інструкцій. Тред виконується певним процесорним паралельно з іншими тредами. Приклад – проект Blue Gene петафлопсного суперкомп'ютера від IBM.

Рівень команд: VLIW-процесори

- Архітектура з довгим командним словом (VLIW – Very Long Instruction Word) бере свій початок від *паралельних мікропрограм*, що вже давно застосовувались в обчислювальній техніці.
- VLIW-команди включають кілька полів (деякі можуть бути порожніми), що відповідають кожне за свою операцію, а всі вони виконуються *одночасно в єдиному циклі.*
- На сьогодні багато *сигнальних процесорів* оснащуються додатковими векторними операціями, що використовують VLIW-подібні довгі інструкції, що «прошивається» в постійному запам’ятовуючому пристрої (наприклад, для виконання швидкого перетворення Фур'є або подібних алгоритмів).
- Звичайно, послідовність настільки складних команд може генеруватися спеціальним *'високоінтелектуальним' компілятором*, що виявляє паралелізм у вихідній програмі і генерує VLIW-інструкції, окремі поля яких можуть бути виконані незалежно один від одного (тобто, фактично, *паралельно*).

«Зерна» – гранули розпаралелювання

- Найбільш загальна схема виконання послідовних і паралельних обчислень наведено нижче, де моменти часу S і E є початком і кінцем завдання.



- *Процесом* називають послідовність команд, що претендує для свого (послідовного) виконання на використання процесора (ядра);
- Тонкими лініями показані дії зі створення процесів і обміну даними між ними, а товстими - безпосередньо виконання процесів (вісь абсцис - час).

Дрібно-зернистий та крупно-зернистий паралелізм

- У разі послідовних обчислень створюється *єдиний процес*; при паралельному виконанні алгоритму необхідні кілька (іноді, десятки / сотні / тисячі) процесів (*паралельних гілок* завдання).
- Довжина послідовності команд паралельного процесу називається *розміром гранули (зерна)*.
- Зазвичай розмір зерна (гранули) становить десятки-сотні тисяч машинних операцій (що на порядки перевищує типовий розмір оператора мов програмування, таких як Fortran або C / C ++).
- Залежно від розмірів гранул говорять про *дрібно-зернистий* і *крупно-зернистому паралелізм* (fine-grained parallelism і coarse-grained parallelism).
- У будь-якому випадку доцільно прагнути до *'крупно-зернистого'* паралелізму як найбільш ефективного способу використання процесорної техніки.

Проста математична модель для оцінки продуктивності (1)

- У простій математичній модель для оцінки можливого підвищення продуктивності при розпаралелюванні обчислень с урахуванням часу обміну даними приймемо t_o - характерний час обміну (операцій пересилання даних між процесами), $t_в$ - час виконання послідовності команд-складових гранули, n - кількість гранул.
- Приймемо (спрощено), що при розпаралелюванні кожна гранула виконується на окремому процесорі, де час обчислення послідовності команд кожної гранули однаковий, а кожен процес вимагає при функціонуванні двох обмінів (на початку і вкінці виконання гранули).

Проста математична модель для оцінки продуктивності (2)

- Тоді послідовний алгоритм виконається (в середньому) складе $T_{посл} = t_в \times n$, а час виконання паралельного алгоритму --$T_{пар} = t_з + 2 \times t_о$.
- Виграш в продуктивності паралельного алгоритму в порівнянні з послідовним (прискорення обчислень) в цьому випадку складає:

$$s = T_{посл} / T_{пар} = (t_з \times n) | (t_з + 2 \times t_о)$$

- Розрахунок кількості процесорів, необхідних для отримання s= 0,1;1 і 10 подано нижче у таблиці (значення n НЕ округлені до цілого):

n	2	3	5	7	10	15	20
$t_o/t_з$=0,1	1,67	2,5	4,17	5,83	8,33	12,5	16,7
$t_o/t_з$=1	0,667	1,0	1,67	2,33	3,33	5,0	6,67
$t_o/t_з$=10	0,095	0,143	0,238	0,333	0,476	0,714	0,952

Рівні паралелізму: програми та завдання

- Вищезазначений висновок про пріоритет більшого значення відношення часу обмінів до часу виконання паралельних процесів приводить до тенденції *укрупнення гранул паралелізму від команд до програм і цілих завдань*. Особливо це важливо для суперком'ютерів.
- *Суперкомп'ютери* – це ексклюзивні обчислювальні системи паралельної дії, що забезпечують обчислення *надвисокої продуктивності.* Вони призначені для вирішення «великих» задач. Прикладами таких задач є задачі, зокрема, з прогнозування змін глобального клімату, обчислення океанських течій, моделювання у складних технічних систем і т.і.
- Довгий час на ринку суперкомп'ютерів домінували системи *векторно-матричної архітектури* з гранулами паралелізму на рівні команд, де найвизначніші досягнення спочатку мала компанія Cray.
- Проте в останні десятиліття лідерство перейняли *мультипроцесорні* та *мультиядерні системи*, з гранулами паралелізму на рівні програм і завдань, що мають значну перевагу у нарощуванні кількості процесорів та засобах забезпечення високих показників продуктивності.